



第三章 栈与队列



第三章（第一部分）栈

提 纲

BUPT



栈的定义



顺序栈



链栈



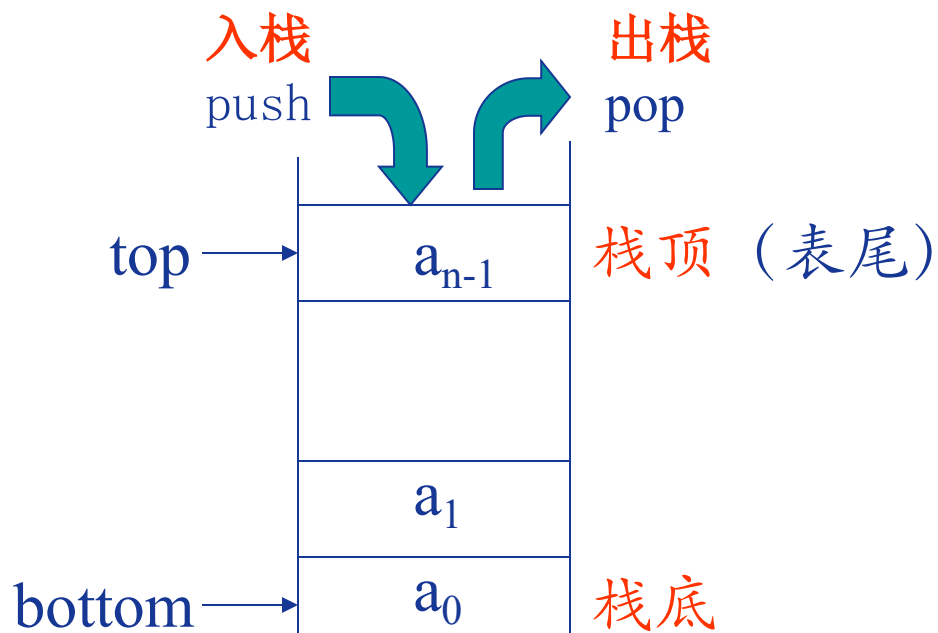
栈的应用

3.1 栈的定义

BUPT

[栈的定义]

栈是一种特殊的线性表，限定插入和删除操作只能在表尾进行。具有后进先出(LIFO—Last In First Out)的特点。



定义在栈结构上的基本运算

- ❖ (1) 生成空栈操作
- ❖ (2) 判栈空函数
- ❖ (3) 数据元素入栈操作
- ❖ (4) 数据元素出栈函数
- ❖ (5) 取栈顶元素函数
- ❖ (6) 置栈空操作
- ❖ (7) 求当前栈元素个数函数

3.2. 顺序栈

BUPT

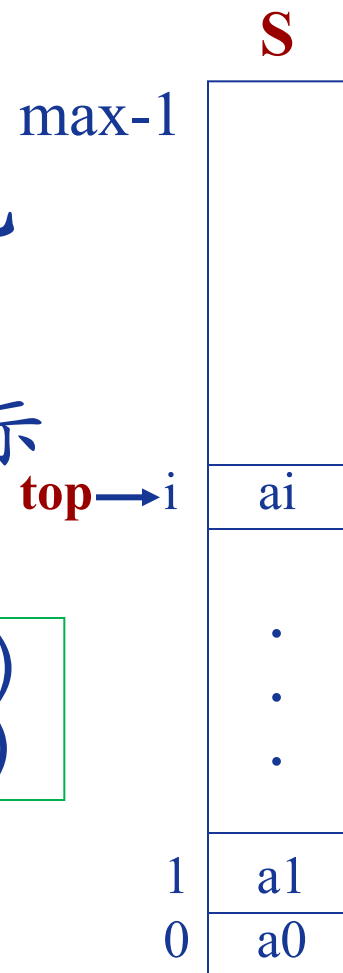
【栈的顺序存储结构】

一个栈独占一组地址连续的存储单元

【类型定义】 数组(栈空间)+栈顶指示

栈空条件 $top == -1$ (此时退栈则下溢)

栈满条件 $top == max-1$ (此时进栈则上溢)



【顺序栈的参考结构】

第一种：直接用数组 `elemtype S[max]; top = -1;`

第二种：

```
struct StackRecord
{
    int capacity; ;//最多容纳元素个数
    int top;
    elemtype *Array;
}
Typedef struct StackRecord * Stack;
```

【顺序栈基本操作】

入栈/进栈

```
void Push(elemtype X, Stack S)
{
    if (S->top == S->Capacity - 1)
        exit("Full!");
    S->Array[++S->top] = X;
}
```

出栈/退栈

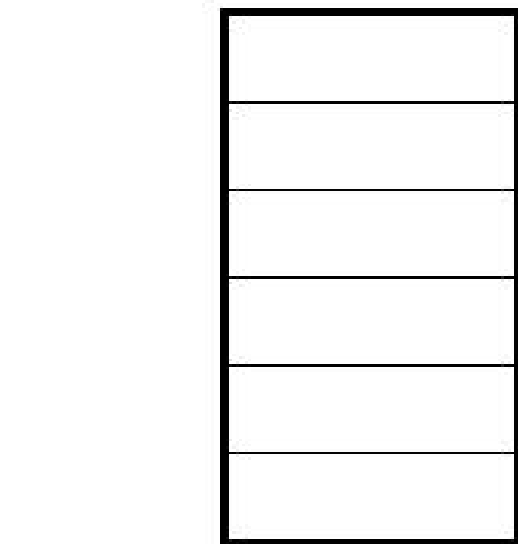
```
void Pop(Stack S)
{
    if (S->top == -1)
        exit("empty!");
    S->top--;
}
```

Stack CreatStack(int max) 栈初始化

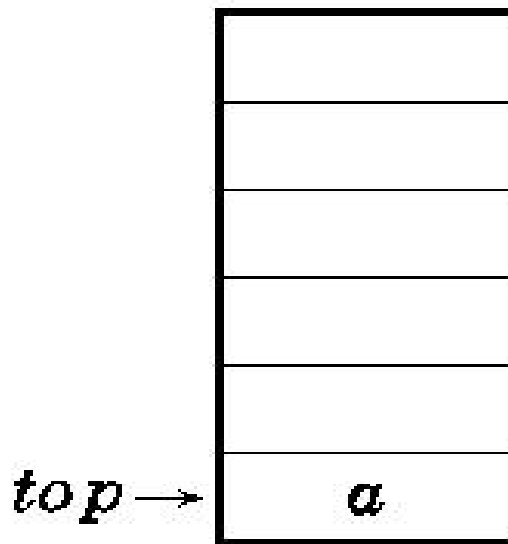
```
{
    Stack S;
    S = malloc(sizeof(struct StackRecord));
    if (S == NULL)
        exit("out of space");
    S->Array = malloc(sizeof(elemtype)* max);
    if (S->Array == NULL)
        exit("out of space");
    S->top = -1;
    S->Capacity = max;
}
```

void DisposeStack(Stack S) 释放栈

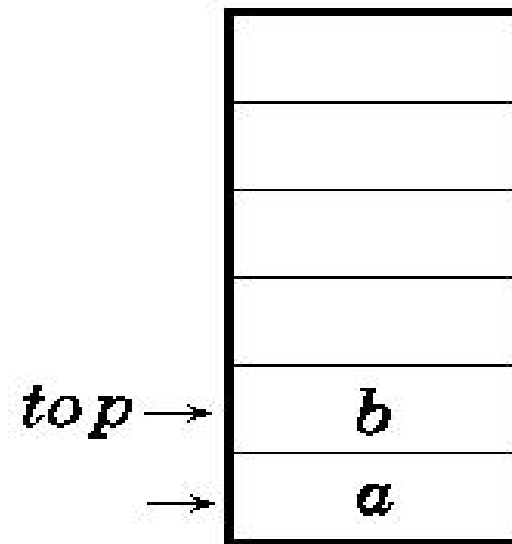
```
{
    if (S != NULL)
    {
        free(S->array);
        free(S);
    }
}
```

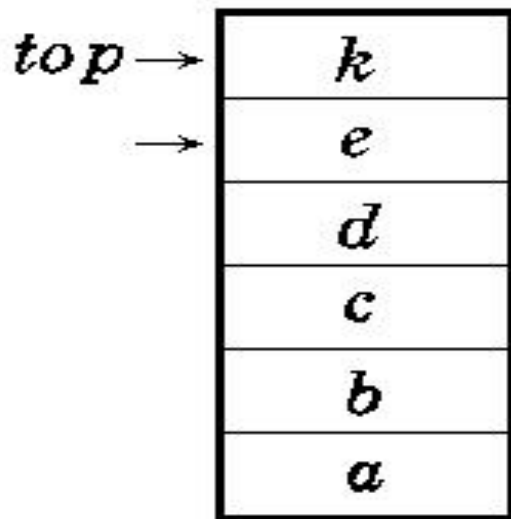
$top \rightarrow$ 空栈



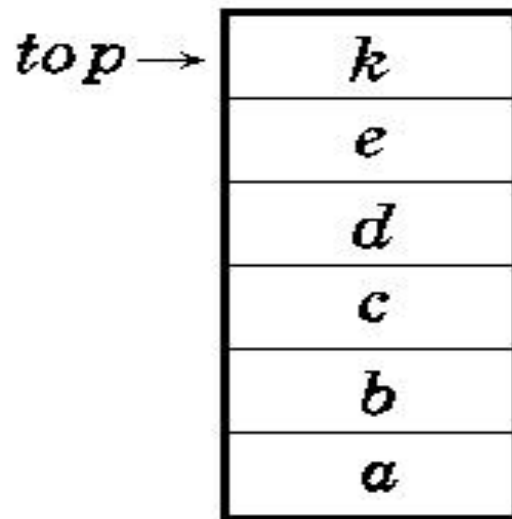
$top \rightarrow$ a 进栈



$top \rightarrow$ b 进栈

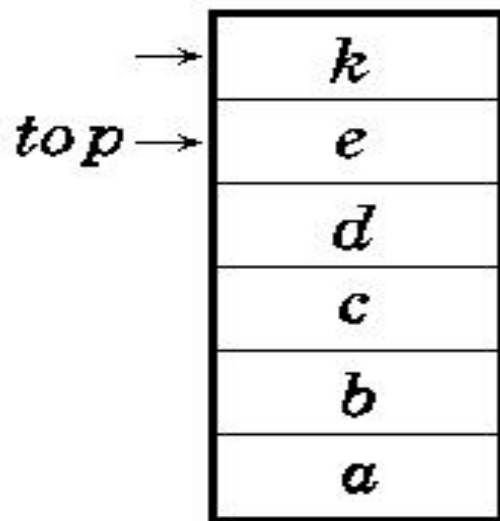


k 进栈

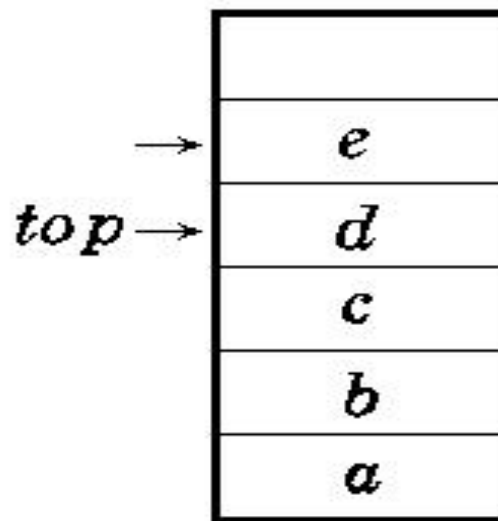


l 进栈溢出

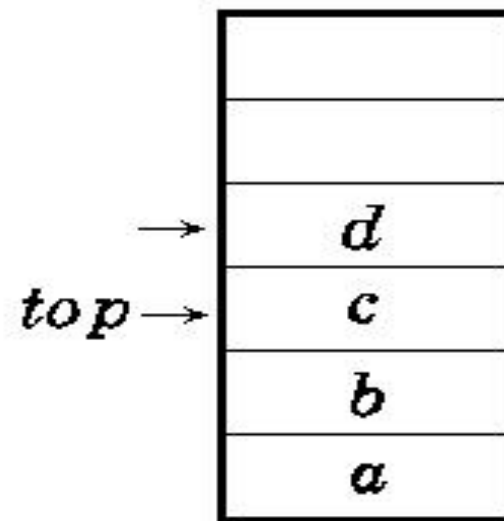
进栈示例



k 退栈



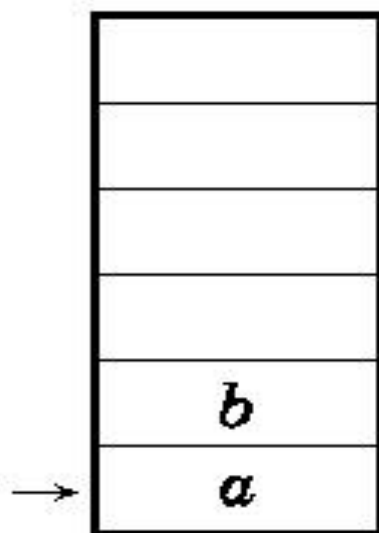
e 退栈



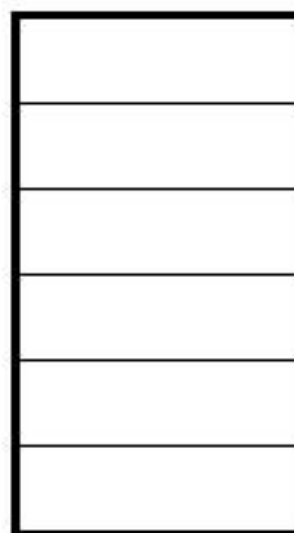
d 退栈

...

...



a 退栈



空栈

退栈示例

有5个元素，其入栈次序为：A，B，C，D，E，在各种可能的出栈次序中，以元素C，D最先出栈（即C第一个且D第二个出栈）的次序有哪几个？

CDEBA, CDBEA, CDBAE

思考：n个元素依次入栈，可得到多少个合法的出栈序列

$$(2n)! / [(n+1)! * n!]$$

不同的出栈序列实际上对应着不同的入栈出栈操作，以1记为入栈，0为出栈。则问题实际上是求n个1和n个0构成的全排列，其中任意一个位置，它及它此前的数中，1的个数要大于等于0的个数。

n个1和n个0构成的全排列数为： $(2n)! / [n! * n!]$

在n个0和n个1构成的2n个数的序列中，假设第一次出现0的个数大于1的个数（即0的个数比1的个数大一）的位置为k，则k为奇数，k之前有相等数目的0和1，各为 $(k-1)/2$ 。若把这k个数，0换成1，1换成0，则原序列唯一对应上一个n+1个1和n-1个0的序列。反之，任意一个由n+1个1和n-1个0构成的序列也唯一的对应一个这样不合要求的序列。由于一一对应，故这样不合要求的序列数实际上等于有n+1个1和n-1个0构成的排列数，即 $(2n)! / (n+1)(n-1)$ 。

因此合法的个数为： $(2n)! / [n! * n!] - (2n)! / [(n+1)! * (n-1)!]$

$$= (2n)! / [(n+1)! * n!]$$

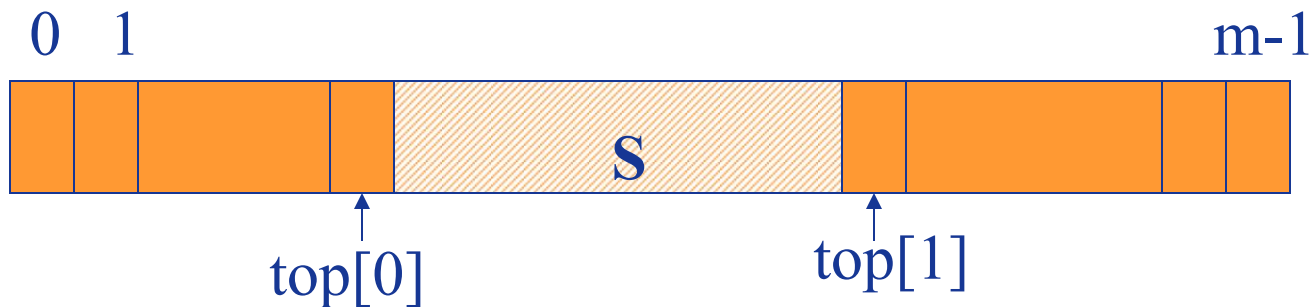
[例] 两个栈共享一组地址连续的存储单元

[类型定义] 数组(栈空间) + 两个栈顶指示

栈满条件: $s.top[0]+1 == s.top[1]$

栈空条件: $s.top[0] == -1$ (栈0)

$s.top[1] == m$ (栈1)



多栈处理 栈浮动技术

n 栈共享一个数组空间 $V[m]$

设立栈顶指针数组 $t[n+1]$ 和

栈底指针数组 $b[n+1]$

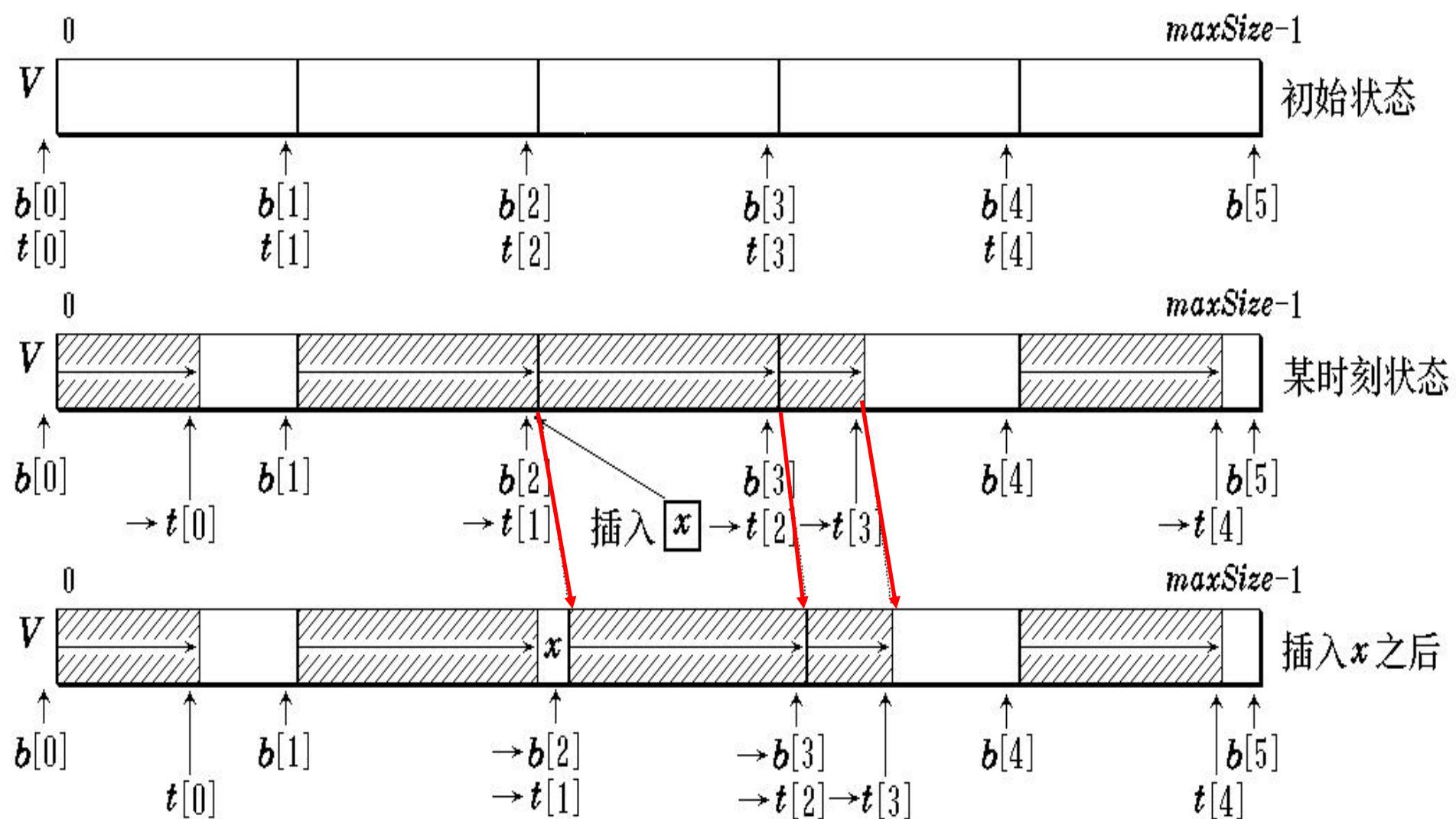
$t[i]$ 和 $b[i]$ 分别指示第 i 个栈的栈顶与栈底

$b[n]$ 作为控制量，指到数组最高下标

各栈初始分配空间 $s = \lfloor m / n \rfloor$

指针初始值 $t[0] = b[0] = -1$ $b[n] = m-1$

$t[i] = b[i] = b[i-1] + s, \quad i = 1, 2, \dots, n-1$



插入新元素时的栈满处理 *StackFull()*

```

template <class Type>
void Push ( const int i, const Type & item ) {
    if ( t [i] == b[i+1] ) StackFull (i);
    else V[++t[i]] = item;    //第i 个栈进栈
}

```

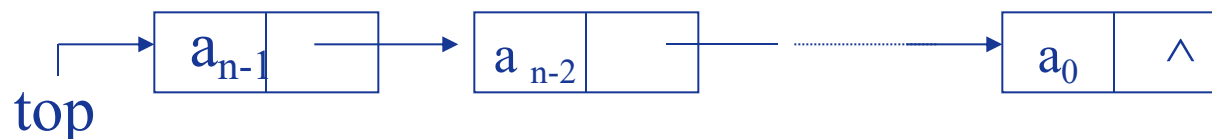
```

template <class Type> Type *Pop ( const i ) {
    if ( t[i] == b[i] )
        { StackEmpty(i); return 0; }
    else return & V[t[i]--];    //第i 个栈出栈
}

```


3.3. 链栈

BUPT



- 链式栈无栈满问题，空间可扩充
- 插入与删除仅在栈顶处执行
- 链式栈的栈顶在链头
- 适合于多栈操作

【链栈结构】

```
struct node
{
    elemtype data;
    struct node *next;
};
typedef struct node Node;
typedef struct node *PtrToNode;
typedef PtrToNode Stack;
typedef PtrToNode Position;
```

【带头结点的单链栈操作】

入栈操作

```
void Push(elemtype X, Stack S)
{
    PtrtoNode tmp;
    tmp = malloc(sizeof(struct node));

    if (tmp == NULL)
        exit("out of space");
    else
    {
        tmp->data = X;
        tmp->next = S->next;
        S->next = tmp;
    }
}
```

出栈操作

```
void Pop(Stack S)
{
    PtrtoNode FirstCell;
    if(S->next != NULL)
    {
        FirstCell = S->next;
        S->next = S->next->next;
        free (FirstCell);
    }
}
```

即带头结点的单链表添加和删除首元结点。

3.4.栈的应用

BUPT

❖ 栈与递归过程

[递归的含义]

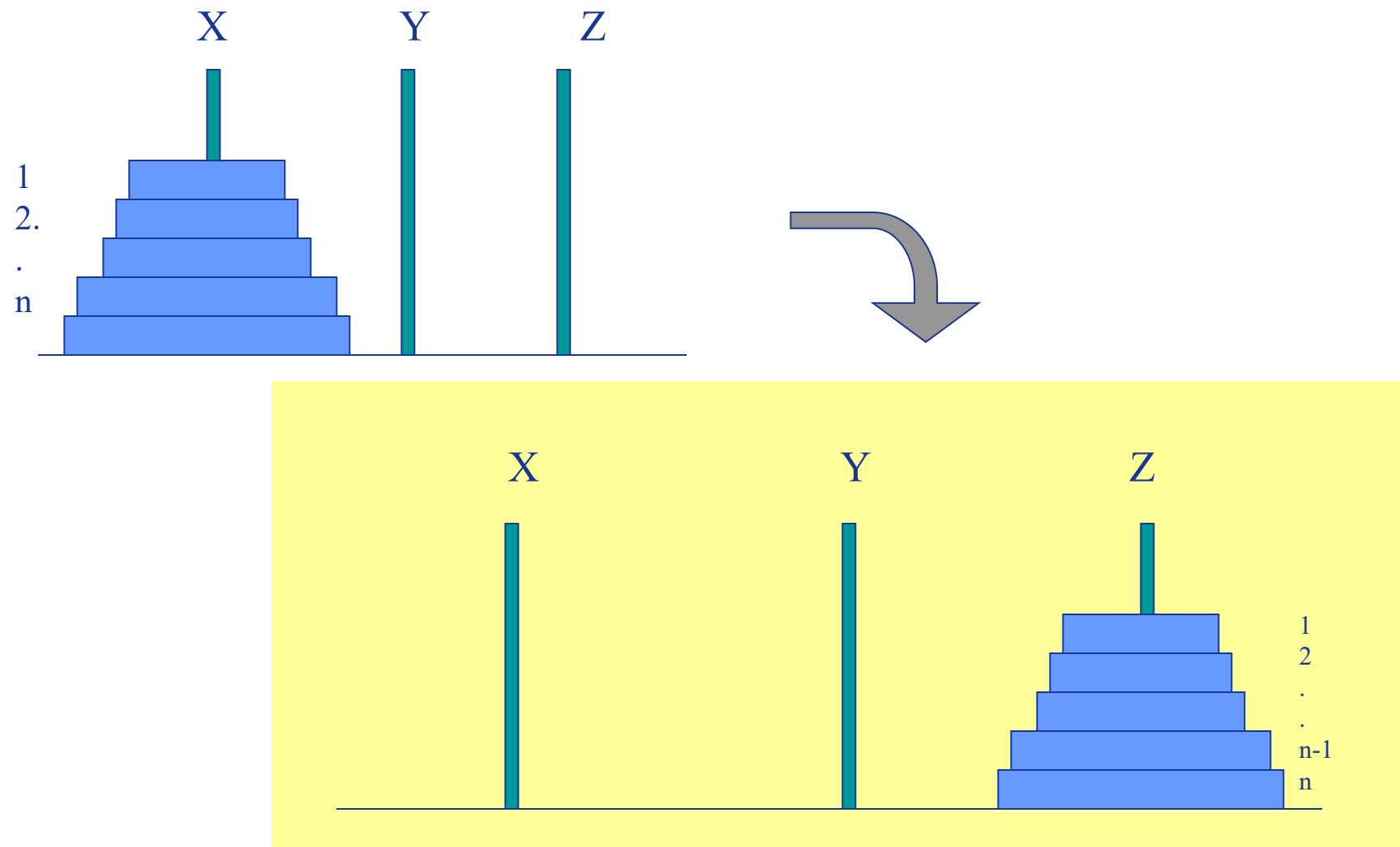
函数、过程或者数据结构的内部又直接或者间接地由自身定义。

[适合于应用递归的场合]

- 规模较大的问题可以化解为规模较小的问题，且二者处理(或定义)的方式一致；
- 当问题规模降低到一定程度时是可以直接求解的(终止条件)

例1. 阶乘 $n! = \begin{cases} 1 & n=0 \\ n \cdot (n-1)! & n>0 \end{cases}$

例2. n阶Hanoi塔问题



[写递归算法应注意的问题]

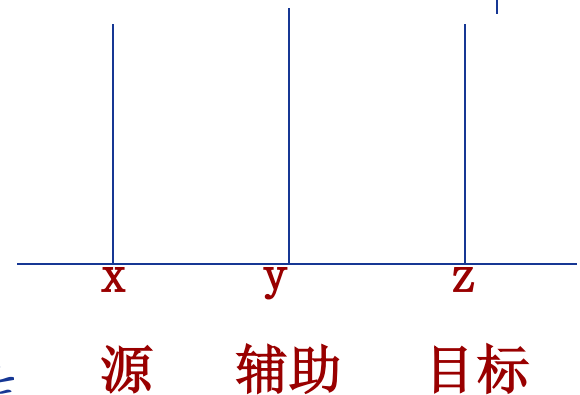
- 递归算法本身不可以作为独立的程序运行，需在其它的程序中设置调用初值，进行调用；
- 递归算法应有出口(终止条件)

例1. 求n!

```
int Factorial(int n)
{
    if (n==0)
        return(1);
    return(n*Factorial(n-1));
}
```

❖ 例2. Hanoi塔问题

```
void Hanoi (int n, int x, int y, int z)
{
    if (n==1)
        Move (x, 1, z) //出口条件
    else{
        Hanoi (n-1, x, z, y);
        Move (x, n, z); //视为简单操作
        Hanoi (n-1, y, x, z);}
}
```



```

void hanoi(3,a,b,c){
    if (3==1)
        move(a,1,c);
    else
    {
        hanoi(2,a,c,b);
        move(a,3,c);
        hanoi(2,b,a,c);
    }
}

```

```

void hanoi(2,a,c,b){
    if (2==1)
        move(a,1,b);
    else
    {
        hanoi(1,a,b,c);
        move(a,2,b);
        hanoi(1,c,a,b);
    }
}

```

```

void hanoi(1,a,b,c){
    if 1=1
        move(a,1,c);
    else {.....}
}

```

```

void hanoi(1,c,a,b){
    if (1==1)
        move(c,1,b);
    else {.....}
}

```

```

void hanoi(1,b,c,a) {
    if (1==1)
        move(b,1,a);
    else {.....}
}

```

```

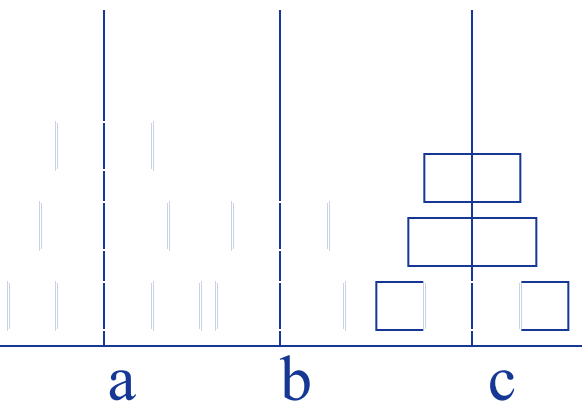
void hanoi(2,b,a,c) {
    if (2==1)
        move(b,1,c);
    else
    {
        hanoi(1,b,c,a);
        move(b,2,c);
        hanoi(1,a,b,c);
    }
}

```

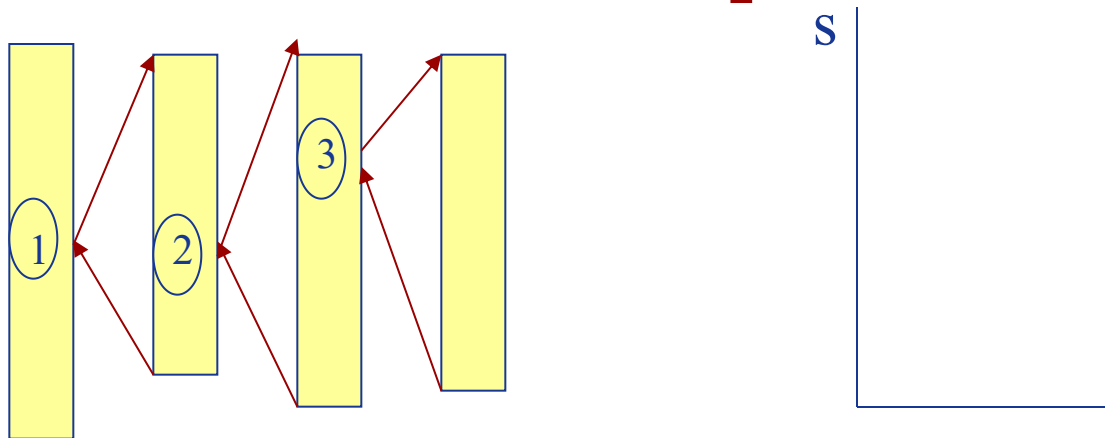
```

void hanoi(1,a,b,c) {
    if (1==1)
        move(a,1,c);
    else {.....}
}

```



[递归算法的实现原理]



- 利用栈，栈中每个元素称为工作记录，分成三个部分：
返回地址 实在参数表(变参和值参) 局部变量
- 发生调用时，保护现场，即当前的工作记录入栈，然后转入被调用的过程
- 一个调用结束时，恢复现场，即若栈不空，则退栈，从退出的返回地址处继续执行下去

[递归时系统工作原理示例]

BUPT

```
int Factorial(int n){  
    L1:  if (n==0)  
    L2:      return(1);  
    L3:  return(n*Factorial(n-1));  
}
```



void main()

```
{  
    .....  
    L0: N=Factorial(3);  
    .....  
}
```



返回地址 n Factorial N

[递归算法的用途]

- 求解递归定义的数学函数
- 在以递归方式定义的数据结构上的运算/操作
- 可用递归方式描述的解决过程

[递归转换为非递归的方法]

1) 采用迭代算法 递归—从顶到底 迭代—从底到顶

例：求n!

```
int fact(int n){  
    m=1;  
    for (i=1;i<=n;i++) m=m*i;  
    return(m); }
```

2) 消除尾递归

BUPT

例：顺序输出单链表中的结点数据(极端不当使用递归)

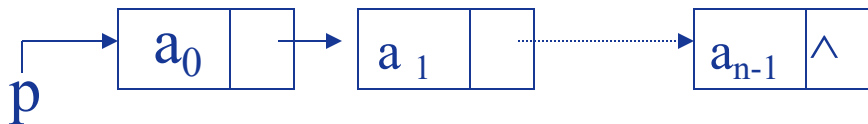
```
void linklist_output(pointer p){  
    IF (p!=NULL)  
    {   write(p->data);  
        linklist_output(p->next);  
    }  
}
```

使用跳转语句：

```
void linklist_output1(pointer p){
```

使用循环语句：

```
void linklist_output2(pointer p){  
    while (p!=NULL)  
    {   write(p->data);  
        p=p->next;  
    }  
}
```



****所有尾递归可以通过将递归调用变成goto，并加上对函数所有参数的赋值语句来消除**

BUPT SCST

3) 利用栈模拟递归—通用方法

BUPT

如果是递归函数，需改写为递归过程

例：求 $n!$

```
void fact(int n; int &f){
```

```
    init_stack(s);
```

```
    0: if (n==0) f=1;
```

```
    else
```

```
    { 1: fact(n-1,f);
```

```
      2: f:=n*f;
```

```
    }
```

```
}
```

top=top+1; s[top].rd=2;
s[top].n=n; s[top].f=f;

n=n-1; goto 0;

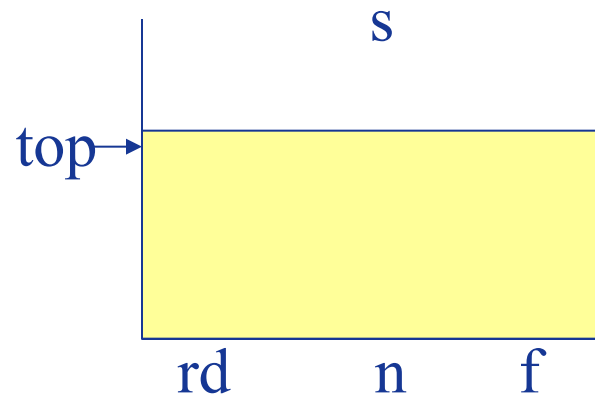
if (top!=0)

{ s[top].f=f; {还原本层变参值}

top=top-1; n=s[top+1].n;

f=s[top+1].f; goto s[top+1].rd }

【自设栈模拟系统工作栈】



改写算法

在程序开头增加栈的初始化语句

改写递归调用语句 { 入栈处理;
确定调用的参数值;
转至调用的开始语句

改写所有递归出口 { 退栈还原参数;
转至返回地址处

```

void fact(int n, int & f)
{
    init_stack(s);
0: if (n==0) f=1;
    { 1: top=top+1;
        s[top].rd=2;
        s[top].n=n;
        s[top].f=f;
        n=n-1; goto 0;
    2: f=n*f }
    if (top!=0)
    { s[top].f=f;
        top=top-1;
        n=s[top+1].n;
        f=s[top+1].f;
        goto s[top+1].rd; }
}

```



```

void fact(int n,int &f)
{
    init_stack(s);
    while (n!=0)
    { top=top+1;
        s[top].n=n;
        n=n-1;}

    f=1;
    while (top!=0)
    { top=top-1;
        n=s[top+1].n;
        f=n*f; }
}

```

注：此时可确定栈中只
存放n值即可。

栈的应用2-整型数简单表达式求值

[前提假设] 表达式语法正确;
表达式结束符为 ‘#’

[算法思想]

OPTR栈—寄存运算符 OPND栈—寄存操作数

1)放入起始符 ‘#’作为OPTR栈底元素

2)依次读入表达式字符,

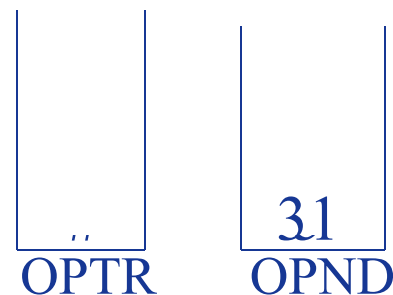
操作数: 进OPND栈;

运算符: 与OPTR.top元素比较优先级后做相应操作;

直至读入 ‘#’且OPTR栈只剩 ‘#’

3)OPND栈底所留元素即为所求

[例] $5+3*(7-2)+11\#$





第三章（第二部分） 队列

提 纲

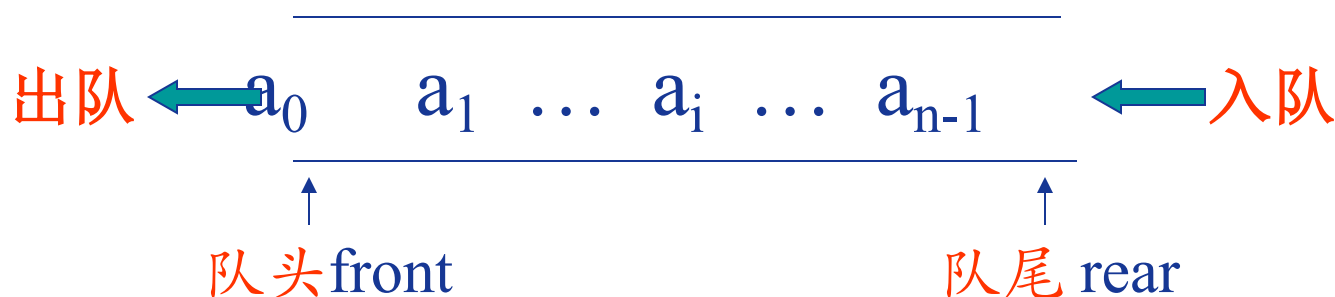
BUPT

- 1 队列的定义
- 2 链队列
- 3 循环队列
- 4 双端队列

3.5 队列的定义

BUPT

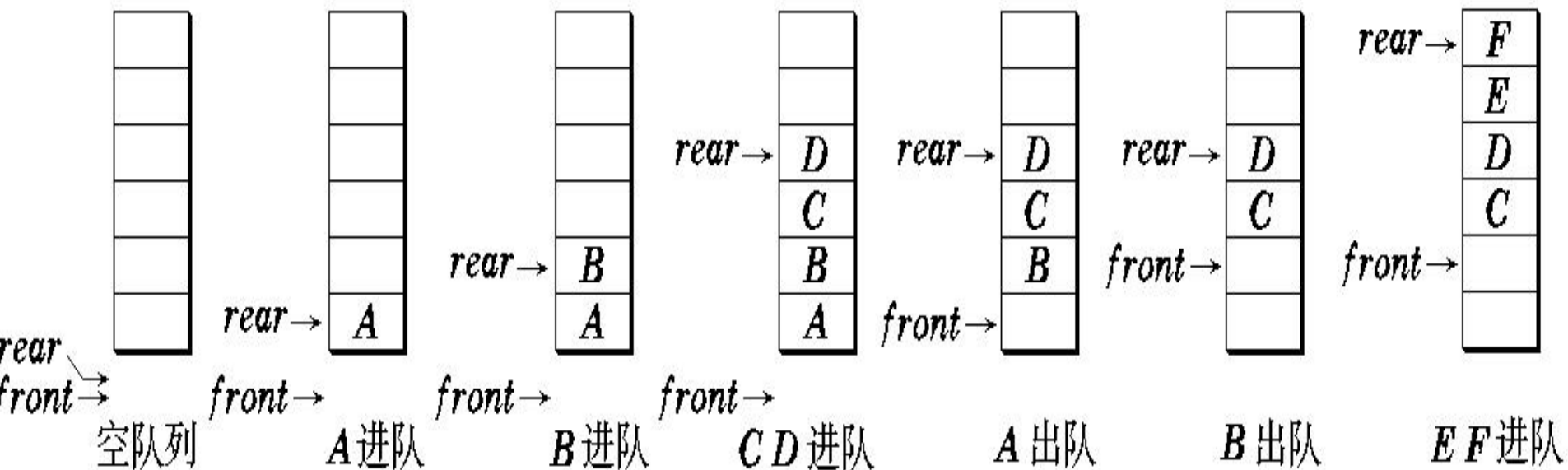
队列是一种特殊的线性表，限定插入和删除操作分别在表的两端进行。具有**先进先出**(FIFO—First In First Out)的特点。



定义在队列结构上的基本运算

- (1)构造空队列操作
- (2)判队空否函数
- (3)元素入队操作
- (4)元素出队函数
- (5)取队头元素函数
- (6) 队列置空操作
- (7)求队中元素个数函数

队列的进队和出队



- 进队时队尾指针先进一 $rear = rear + 1$ ，再将新元素按 $rear$ 指示位置加入。
- 出队时队头指针先进一 $front = front + 1$ ，再将 $front$ 指示的元素取出。
- 队满时再进队将溢出出错；队空时再出队将队空处理。

思考：可否用两个栈实现一个队列？如何实现？

利用两个栈s1、s2（等容量）模拟一个队列,实现队列的插入，删除以及判队空运算

- ❖ 栈的特点是后进先出，队列的特点是先进先出。初始时设栈s1和栈s2均为空。
- ❖ （1）用栈s1和s2模拟队列的输入：设s1和s2容量相等。分以下三种情况讨论：若s1未滿，则元素入s1栈；若s1滿，s2空，则将s1全部元素退栈，再压栈入s2，之后元素入s1栈；若s1滿，s2不空（已有出队列元素），则不能入队。
- ❖ （2）用栈s1和s2模拟队列出队（删除）：若栈s2不空，退栈，即是队列的出队；若s2为空且s1不空，则将s1栈中全部元素退栈，并依次压入s2中，s2栈顶元素退栈，这就是相当于队列的出队。若栈s1为空并且s2也为空，队列空，不能出队。
- ❖ （3）判队空 若栈s1为空并且s2也为空，才是队列空。

讨论：s1和s2容量之和是队列的最大容量。其操作是，s1栈滿后，全部退栈并压栈入s2（设s1和s2容量相等）。再入栈s1直至s1滿。这相当队列元素“入队”完毕。出队时，s2退栈完毕后，s1栈中元素依次退栈到s2，s2退栈完毕，相当于队列中全部元素出队。

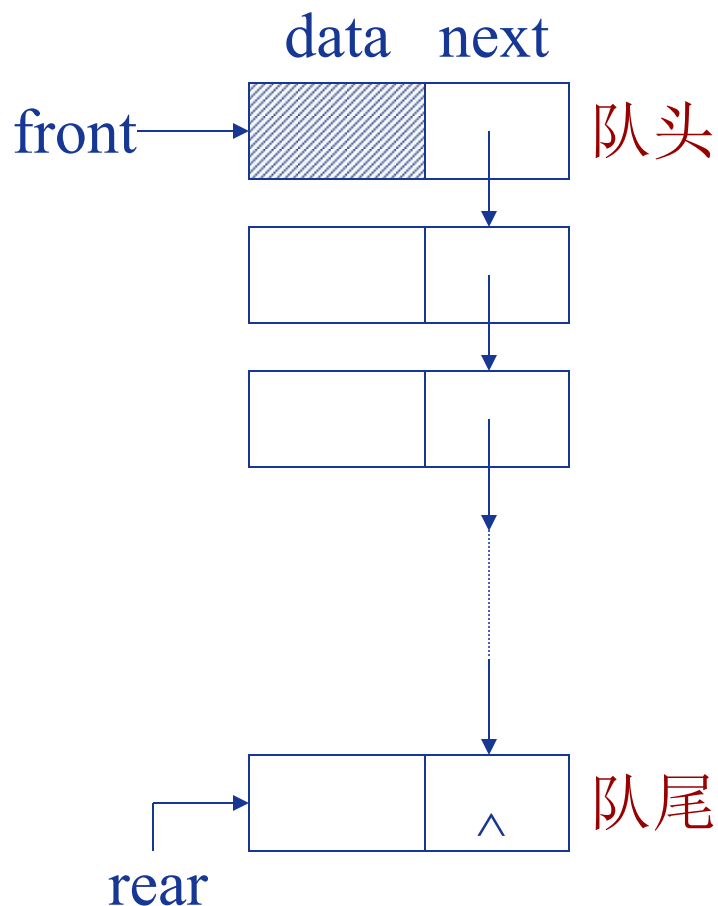
在栈s2不空情况下，若要求入队操作，只要s1不滿，就可压入s1中。若s1滿和s2不空状态下要求队列的入队时，按出错处理。

3.6 链队列

BUPT

用单链表表示的队列

类型定义：队头指针 + 队尾指针

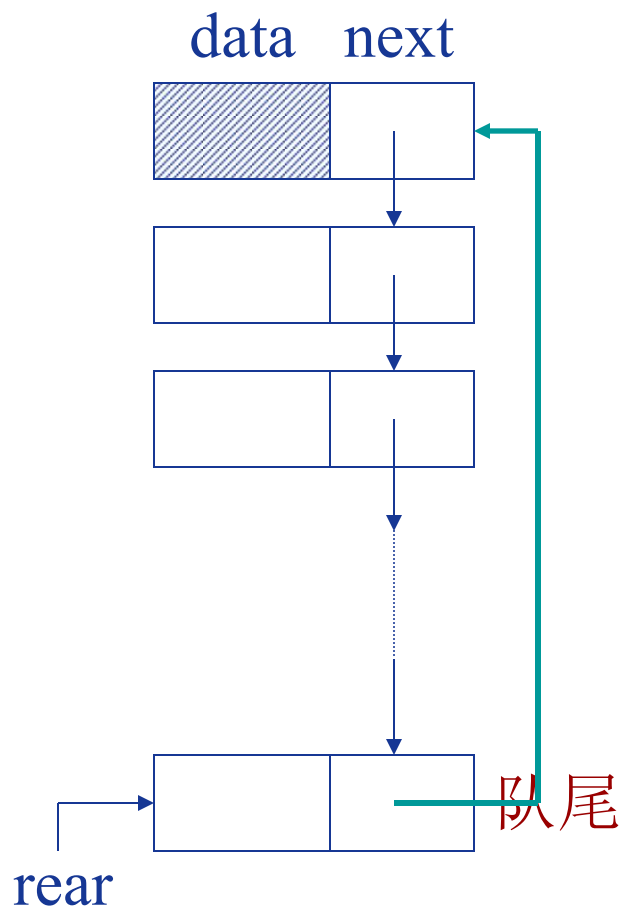


- ❖ 队头在链头，队尾在链尾。
- ❖ 链式队列在进队时无队满问题，但有队空问题。
- ❖ 队空条件为

$front \rightarrow next == NULL$

用单循环链表定义的队列

类型定义：队尾指针



3.7 循环队列

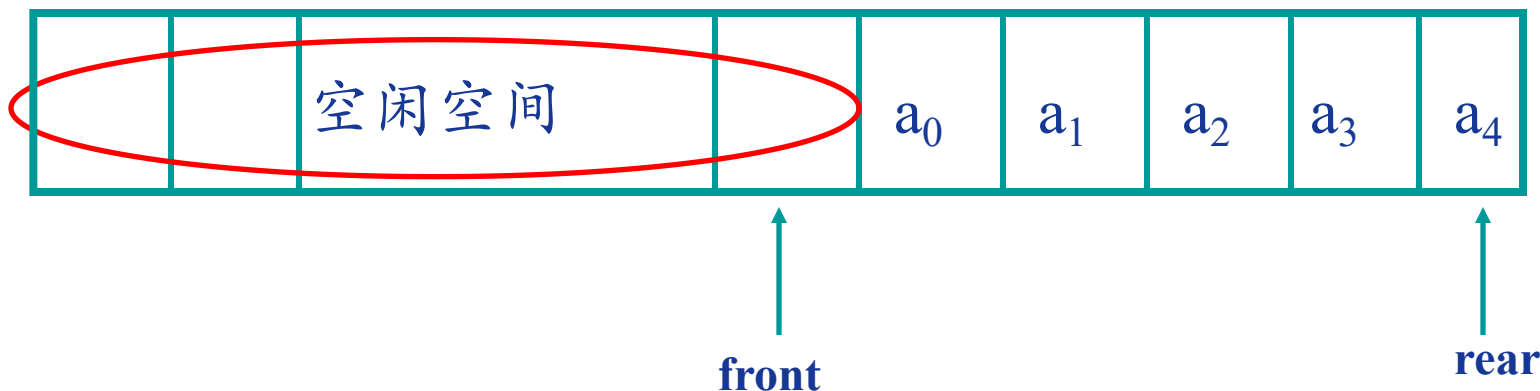
BUPT

一般用法的顺序存储结构之缺陷

出队列操作后大量移动数据或存在“假上溢”现象：即存在空闲空间但无法入队。

解决方法

- ❖ 平移元素法，当发生假溢出时，就把整个队列的元素平移到存储区的首部，然后再插入新元素。这种方法需移动大量的元素，因而效率是很低的。
- ❖ 循环队列

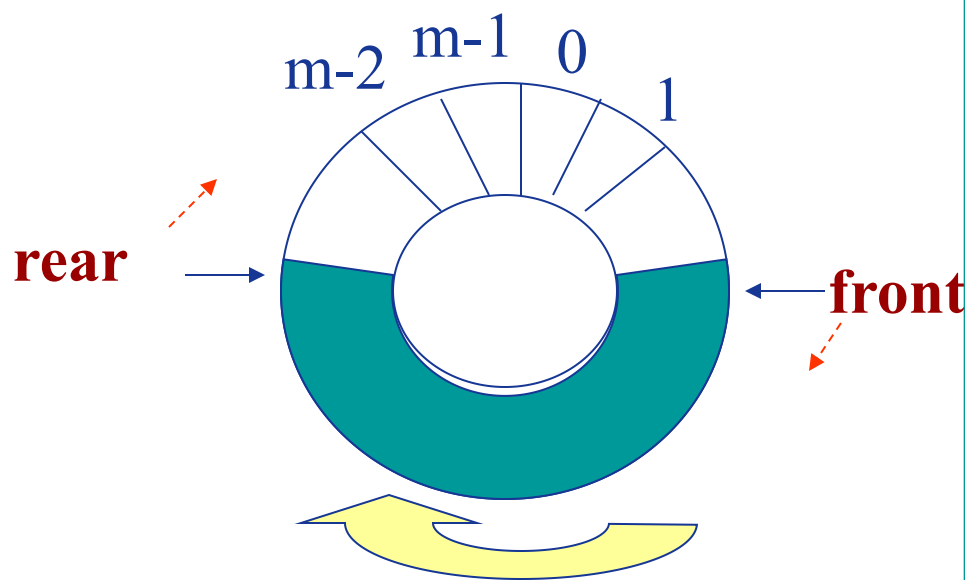


BUPT SCST

[循环队列] 将顺序队列的存储区假想为一个环状空间

[类型定义] 一维数组(队列空间) + 头指针 + 尾指针

队头指针front指向队头元素的前一位置，而队尾指针rear指向队尾元素。

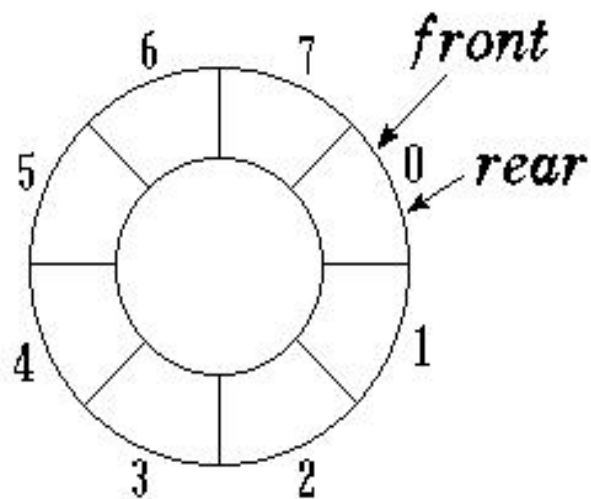


```
typedef struct QueueRecord
{
    elemtype *Array;
    int front; /*队头指针*/
    int rear; /*队尾指针*/
    int size; //当前队列元素个数
    int Capacity; //最多容纳个数
} Queue;
Queue *Q;
```

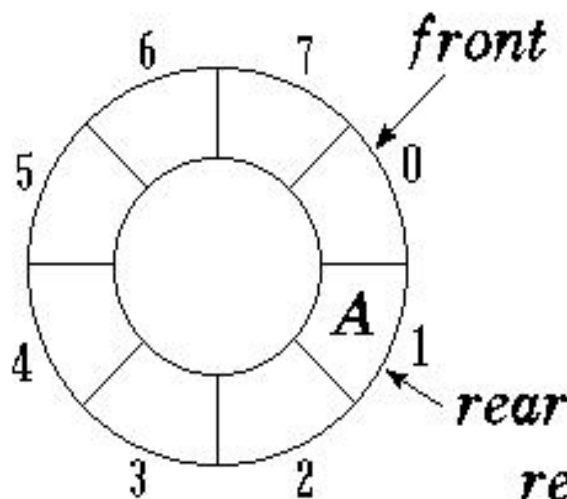
优点： 不需要移动元素，操作效率高，空间的利用率也很高。

循环队列的进队和出队实例

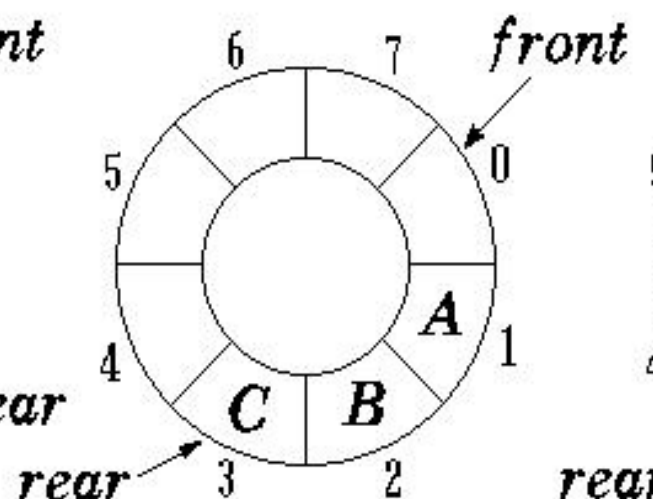
BUPT



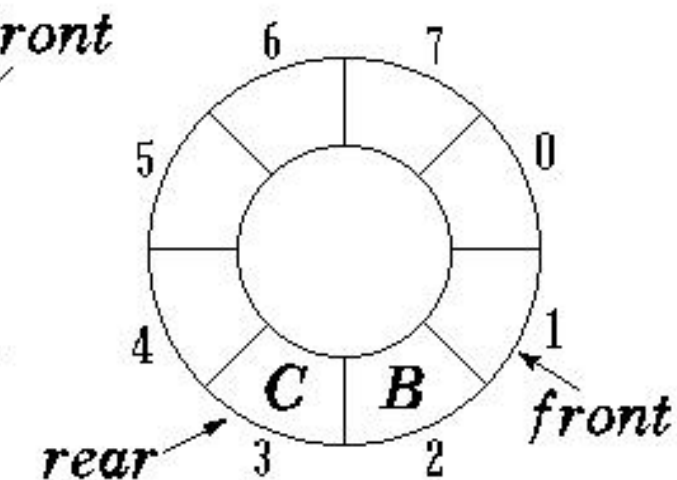
空队列



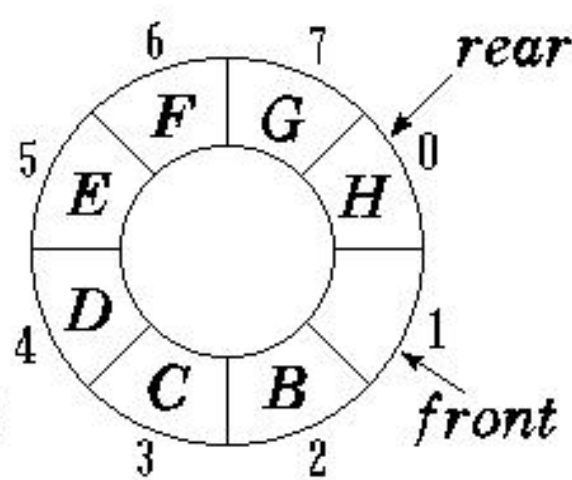
A 进队



BC 进队



A 出队



DEFGH 进队

存储队列的数组被当作首尾相接的表处理。

为解决如何判断队列的满与空问题：牺牲一个存储单元。

队头、队尾指针加1时从 $Q \rightarrow \text{Capacity} - 1$ (上一页中 $m-1$) 进到0，可用语言的取模(余数)运算实现。

出队: $Q \rightarrow \text{front} = (Q \rightarrow \text{front} + 1) \% Q \rightarrow \text{Capacity}; Q \rightarrow \text{size}--;$

入队: $Q \rightarrow \text{rear} = (Q \rightarrow \text{rear} + 1) \% Q \rightarrow \text{Capacity}; Q \rightarrow \text{Array}[Q \rightarrow \text{rear}] = x;$

$Q \rightarrow \text{size}++;$

队列初始化: $Q \rightarrow \text{front} = Q \rightarrow \text{rear} = 0; Q \rightarrow \text{size} = 0;$

队空条件: ① $Q \rightarrow \text{front} == Q \rightarrow \text{rear};$ 或者 ② $Q \rightarrow \text{size} == 0;$

队满条件: ① $(Q \rightarrow \text{rear} + 1) \% Q \rightarrow \text{Capacity} == Q \rightarrow \text{front}$ 或者 ② $Q \rightarrow \text{size} == Q \rightarrow \text{Capacity};$

思考：循环队列中当前元素的个数为多少？

当前元素个数 $Q \rightarrow \text{size}$ ，或者 $(Q \rightarrow \text{rear} - Q \rightarrow \text{front} + Q \rightarrow \text{Capacity}) \% Q \rightarrow \text{Capacity}$

[例]假设以数组sq[8]存放循环队列元素,变量front指向队头元素的前一位置,变量rear指向队尾元素,如用A和D分别表示入队和出队操作。

请给出：执行操作序列 $A^3D^1A^5D^2A^1D^2A^4$ 时的状态。

队空的初始条件：front=rear=0;

执行操作 A^3 后，rear=3; // A^3 表示三次入队操作

执行操作 D^1 后，front=1; // D^1 表示一次出队操作

执行操作 A^5 后，rear=0;

执行操作 D^2 后，front=3;

执行操作 A^1 后，rear=1;

执行操作 D^2 后，front=5;

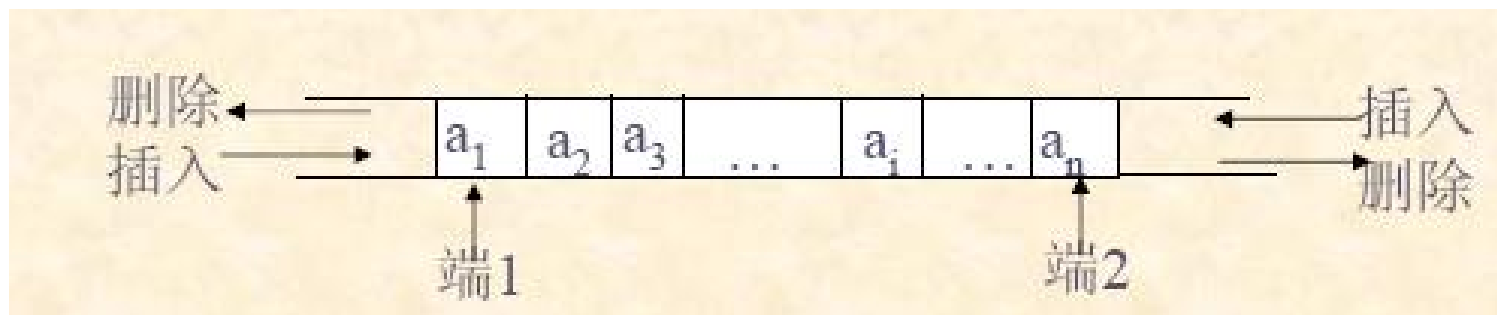
执行操作 A^4 后，溢出。因为执行 A^3 后，rear=4，这时队满，若再执行A操作，则出错。

3.8 双端队列

BUPT

[双端队列] 限定插入和删除操作在线性表的**两端**进行。

可以将它看成是栈底连在一起的两个栈，但它与两个栈共享存储空间是不同的。共享存储空间中的两个栈的栈顶指针是向两端扩展的，因而每个栈只需一个指针；而双端队列允许两端进行插入和删除元素，因而每个端点必须设立两个指针。



❖ 实际应用中的特殊队列

- 输出受限的双端队列（即一个端点允许插入和删除，另一个端点只允许插入）；
- 输入受限的双端队列（即一个端点允许插入和删除，另一个端点只允许删除）。
- 如果限定双端队列从某个端点插入的元素只能从该端点删除，则双端队列就蜕变为两个栈底相邻接的栈了。

如果允许在循环队列的**两端都可以进行插入和删除操作**，其结构如下：

#define M 队列可能达到的最大长度

typedef struct

{ elemtype data[M];

int front,rear;

} cycqueue;

请写出“从队尾删除”和“从队头插入”的算法。

elemtype delqueue (cycqueue Q)

void enqueue (cycqueue Q, elemtp x)

[分析] 用一维数组 $v[\text{MAX}]$ 实现循环队列，其中 MAX 是队列长度。设队头指针 front 和队尾指针 rear ，约定 front 指向队头元素的前一位置， rear 指向队尾元素。定义 $\text{front}=\text{rear}$ 时为队空， $(\text{rear}+1)\% \text{MAX}=\text{front}$ 为队满。约定队头端入队向下标小的方向发展，队尾端入队向下标大的方向发展。

elemtype delqueue (cycqueue Q)

//Q是如上定义的循环队列，本算法实现从队尾删除，若删除成功，返回被删除元素，否则给出出错信息。

```
{  
    if (Q.front==Q.rear)  
    {  
        printf(“队列空” ); exit(0);  
    }  
    Q.rear=(Q.rear-1+MAX)%MAX; //修改队尾指针。  
    return(Q.data[(Q.rear+1+MAX)%MAX]); //返回出队元素。  
}
```

```
void enqueue (cycqueue Q, elemtp x)
```

```
// Q是顺序存储的循环队列，本算法实现“从队头插入”元素x。
```

```
{
```

```
    if (Q.rear==(Q.front-1+M)%M)
```

```
    { printf(“队满” );
```

```
        exit(0);
```

```
    }
```

```
    Q.data[Q.front]=x;    //x 入队列
```

```
    Q.front=(Q.front-1+M)%M; //修改队头指针。
```

```
}
```

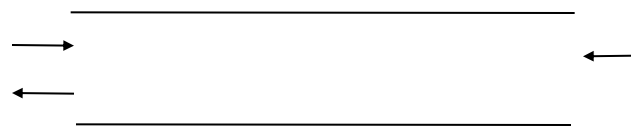
若以1、2、3、4作为双端队列的输入序列，
试分别求出以下条件的输出序列：

- (1) 能由输入受限的双端队列得到，但不能由输出受限的双端队列得到的输出序列；
- (2) 能由输出受限的双端队列得到，但不能由输入受限的双端队列得到的输出序列；
- (3) 既不能由输入受限的双端队列得到，也不能由输出受限的双端队列得到的输出序列。

解答：

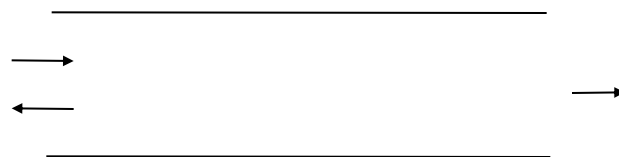
1. 输出受限双端队列不能得到的输出序列有：

4 1 3 2 4 2 3 1



2. 输入受限双端队列不能得到的输出序列有：

4 2 1 3 4 2 3 1



本章作业

BUPT

1. 试推导求解 n 阶hanoi塔问题至少要执行的移动操作 move 次数。
2. 试证明：若借助栈由输入序列 $1, 2, \dots, n$ 得到输出序列为 $P_1, P_2, \dots, P_n$ （它是输入序列的一个排列），则在输出序列中不可能出现这样的情形：存在着 $i < j < k$, 使 $P_j < P_k < P_i$ 。

3. 假设以I和O分别表示入栈和出栈操作。栈的初态和终态均为空，入栈和出栈的操作序列可表示为仅由I和O组成的序列，称可以操作的序列为合法序列，否则称为非法序列。

(1) 下面所示的序列中哪些是合法的？

- A. IOIIIOIOO B. IOOIIOIIO C. IIIIOIOIO
D. IIIOOIOOO

(2) 通过对(1)的分析，写出一个算法，形式如下：**bool Judge(char A[])**，判定所给的操作序列是否合法。若合法，返回true，否则返回false（假定被判定的操作序列已存入一维数组A[]中）。

实验报告（二）

BUPT

- 1. 括弧匹配问题：编写程序，从标准输入得到的一个缺失左括号的表达式，输出补全括号之后的中序表达式。例如：输入 $1+2)*3-4)*5-6)))$ ；则程序应该输出 $((1+2)*((3-4)*(5-6)))$
- 2. 判别回文字符串：正读和反读都一样的字符串称为回文字符串。编写程序，在键盘上输入一个字符串，以“#”作为结束标志，判别它是否为回文字符串。要求：采用栈和队列来实现

说明：可把两题实验报告写在一起